

Qt 프로젝트 보고서

작성자: Doyeon Park

작성일: 2026.09.14

목차

1. 프로젝트 개요
2. 프로젝트 구조
3. ROS2 노드 통신
4. UI 구성과 조작
5. 미로 생성과 게임 규칙
 6. 예외 처리
 7. 실행 결과

1. 프로젝트 개요

이 프로그램은 C++, Qt5, ROS2 로 만든 로봇 미로 탈출 게임이다. 플레이어(사용자)는 키보드 방향키로 로봇을 움직여 노란 열쇠 1 ~ 3 개(난이도에 따라 달라짐)를 모은 뒤 출구로 탈출해야 하는 게임이다. 탈출 도중 적과 만나서 부딪히거나, 제한시간안에 탈출을 못하거나, 로봇의 배터리가 소진되면 게임이 종료된다.

이 프로젝트를 진행할 때 화면과 게임 규칙을 서로 다른 ROS2 노드로 나누었다. 화면은 입력을 명령으로 전달하고, 게임 엔진이 계산한 상태를 그린다. 미로, 위치, 점수, 시간, 승패 판단은 엔진 노드가 관리하기 때문에 두 노드 사이의 실제 토픽과 서비스 통신을 확인할 수 있다.

2. 프로젝트 구조

구성	주요 파일	역할
화면	ui/mainwindow.ui src/mainwindow.cpp	화면 배치, 키 입력, 게임 상태 표시
ROS 연결	src/gui_ros_bridge.cpp src/game_engine_node.cpp	토픽 발행·구독과 재시작 서비스
게임 규칙	src/game_engine.cpp src/game_map.cpp	미로 생성, 이동, 적 추적, 승패 판정
인터페이스	msg/*.msg srv/RestartGame.srv	명령·상태 메시지와 재시작 요청
실행 시험	launch/robot_maze.launch.py test/test_game_engine.cpp	두 노드 실행과 게임 규칙 단위 시험

패키지 이름은 robot_maze_game 이다. launch 파일은 game_gui_node 와 game_engine_node 를 /robot_maze 네임 스페이스에서 함께 실행한다. 게임 규칙 클래스는 Qt 와 ROS2 의존성 없이 분리되어 있어 단위 시험에서 직접 사용할 수 있다.

3. ROS2 노드 통신

통신	방향	내용
player_command 토픽	GUI → 엔진	시작, 상하좌우 이동, 일시정지&재개 명령과 난이도
game_state 토픽	엔진 → GUI	미로 타일, 객체 위치, 점수, 남은 시간, 배터리, 상태
restart_game 서비스	GUI → 엔진	같은 미로 재시작 또는 새 미로 생성 요청과 결과

엔진은 시작 시 상태를 한 번 발행하고, 명령을 처리하거나 게임이 진행되는 동안 주기적으로 상태를 다시 발행한다. GUI 는 받은 상태만 표시한다. 재시작은 결과가 필요한 요청이므로 서비스로 구현했다. 상태 토픽에는 reliable 과 transient local QoS 가 지정되어 있다.

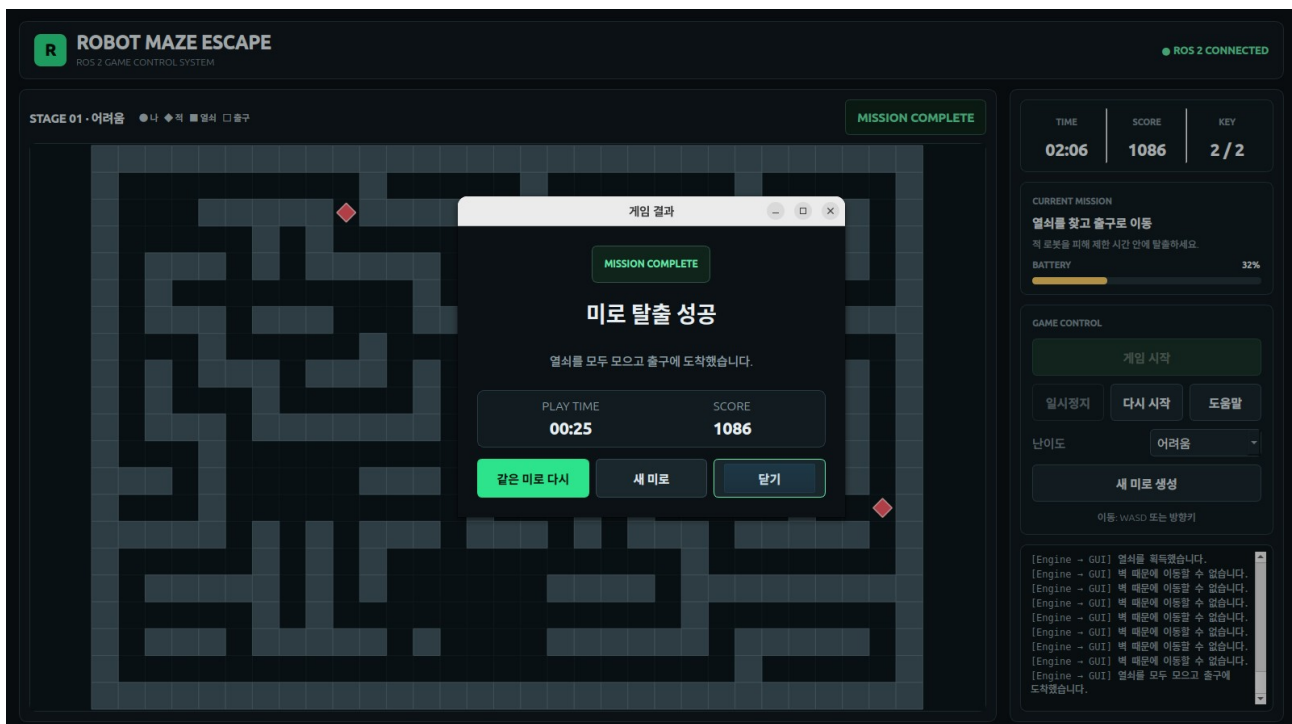
4. UI 구성과 조작

Qt Designer 로 만든 메인 화면은 미로 영역과 정보&조작 영역으로 구성된다. 미로는 QGraphicsScene 에 벽, 플레이어, 열쇠, 적 로봇, 출구를 그린다. 오른쪽에는 남은 시간, 점수 획득한 열쇠, 배터리, 현재 임무, 난이도 선택, 게임 버튼과 이벤트 로그를 배치했다.



WASD 또는 방향키로 이동 가능하고 P 또는 Space, 또는 일시정지 버튼을 눌러 일시정지 시킬 수 있다. 이동 키를 누르고 있으면 150ms 간격으로 명령을 전송하여 키를 꼭 누르고 있을때 끊임없이 이동할 수 있게 구현하였다.

다시 시작 버튼은 현재 시드를 사용하여 같은 미로를 초기화하고, 새 미로 생성 버튼은 다른 시드를 요청한다. 게임이 승리 또는 패배 상태가 되면 결과 대화상자에 사유와 점수, 경과 시간이 표시된다.



GUI는 처음에 엔진 상태를 기다리며 버튼을 비활성화한다. 상태를 수신하면 연결 표시와 버튼 상태를 갱신하게 된다. 전달받은 타일 배열의 길이가 화면 크기와 일치하지 않으면 오류를 표시하게 하였다.

5. 미로 생성과 게임 규칙

5.1 미로 생성과 경로 탐색

GameMap 은 기본 31 X 21 격자를 벽으로 채운 뒤 깊이 우선 탐색 방식으로 통로를 만든다. 이어서 방을 추가하고 일부 벽을 열어 순환 경로를 만든다. 출발점은 (1, 1)이며, 너비 우선 탐색으로 출발점에서 가장 멀리 있는 바닥 칸을 출구로 정한다. 같은 시드로 생성하면 같은 미로가 만들어지도록 난수 엔진을 사용하였다.

경로 길이는 너비 우선 탐색으로 계산한다. 이 값은 출발점과 출구의 연결성을 확인할 때뿐만이 아니라 적 로봇이 플레이어에게 가까워지는 다음 칸을 고를 때에도 사용한다. 참고로 적은 벽이나 다른 적이 있는 칸으로 이동하지 않는다.

5.2 승패와 점수

GameEngine 은 준비, 진행, 일시정지, 승리, 패배, 오류 상태를 구분한다. 진행 중에만 이동과 시간이 갱신된다. 벽을 향한 입력은 이동하지 않으며, 점수와 배터리도 유지된다. 이동에 성공하면 점수가 1 점 증가하고, 열쇠를 얻으면 100 점이 추가된다. 열쇠 두 개를 모두 획득한 뒤 출구에 도착하면 500 점과 남은 시간 1 초당 2 점이 추가되고 승리하게 된다.

만약 적과 충돌하거나, 제한 시간이 끝나거나, 배터리가 0 이되면 패배하게 된다. 적은 플레이어 쪽으로 한칸씩 이동하며, 열쇠와 적은 시작점&출구에서 일정거리 떨어져있으며 이동가능한 칸에 배치된다.

5.3 난이도 설정

난이도	제한시간	적 수	적 이동 간격	배터리 1 소모당 이동 가능한 횟수
쉬움	240 초	1	900ms	4 회
보통	180 초	2	650ms	3 회
어려움	150 초	3	450ms	2 회

난이도는 시작 명령 또는 재시작 요청에 포함되며 설정 값은 엔진에서 검증한 후 게임 생성에 적용된다.

6. 예외 처리

잘못된 미로 크기는 생성자에서 거부하고, 생성된 미로에 출발점부터 출구까지 경로가 있는지 확인한다. 열쇠와 적을 배치할 공간이 부족하면 오류로 처리한다. 알 수 없는 이동 명령은 엔진 노드에서 경고만 기록하고 무시한다.

GUI 는 재시작 서비스가 준비되지 않았거나 응답 처리 중 예외가 발생하면 실패 메시지를 표시한다. 엔진 상태를 일정 시간 받지 못하면 연결 실패를 화면에 표시한다. 이 처리는 게임 규칙 오류와 통신 오류를 사용자에게 구별해서 알려주기 위함이다.

7. 실행 결과

미로, 플레이어, 열쇠, 적, 출구, 시간&점수&배터리&로그 패널이 표시된다. 코드에서도 입력을 player_command 로 보내고 받은 game_state 를 화면에 반영하는 경로를 확인하였다.

이번 프로젝트는 Qt GUI 와 ROS2 게임 엔진을 분리하고, 토픽 두 개와 서비스 한 개로 입력, 상태, 재시작을 연결하였으며 미로 생성, 경로 탐색, 적 추적, 난이도 별 제한, 승패 판정을 게임 규칙 클래스로 분리해 구조를 명확히 했다.